Linux Process Creation Using fork(), exec(), and wait() System Calls

-2520030106

2520030561

2520030183

2520030480

OSSP (Operating Systems And Systems Programming) - 25CS2104E

1. Develop a C program that demonstrates how a Linux operating system executes a command entered by a user that 1. Accept a Linux command as input. 2. Create a child process using fork(). 3. Execute the command in the child process using an appropriate exec() system call. 4. Allow the parent process to wait for the child using wait (). 5. Display the Process ID (PID) of both parent and child processes.

Using Linux terminal commands (uname, lscpu, lsblk, ps, top), investigate the relationship between hardware resources and operating system services. Prepare a report explaining how the OS abstracts CPU, memory, storage, and I/O devices.

Aim

To develop a C program that accepts a Linux command from the user, creates a child process using the fork() system call, executes the command using the execvp() system call, synchronizes the parent process using wait(), and displays the Process IDs (PIDs) of both the parent and child processes.

Requirements

- Ubuntu Linux
- GCC Compiler
- C Programming Language
- Linux Terminal

Theory

Linux is a multitasking operating system that manages multiple processes efficiently. A process is a program in execution. Linux provides several system calls for process creation and management.

- **fork()** creates a new child process by duplicating the parent process.

- **execvp()** replaces the child process with a new program specified by the user.
- **wait()** makes the parent process wait until the child process completes execution.
- **getpid()** returns the Process ID (PID) of the current process.
- **getppid()** returns the Parent Process ID (PPID) of the current process.

These system calls are essential for understanding process management in Linux.

Algorithm

1. Start the program.
2. Include the required header files.
3. Accept a Linux command from the user.
4. Remove the newline character from the input.
5. Split the command into individual arguments.
6. Create a child process using fork().
7. If the child process is created:
    - Display the Child PID and Parent PID.
    - Execute the command using execvp().
8. If the current process is the parent:
    - Display the Parent PID and Child PID.
    - Wait for the child process using wait().
9. Display the completion message.
10. Stop the program.

Code

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <sys/types.h>
#include <sys/wait.h>

#define MAX_ARGS 20
```

```
int main()
{
    char input[100];
    char *args[MAX_ARGS];

    // Read Linux command
    printf("Enter a Linux command: ");
    fgets(input, sizeof(input), stdin);

    // Remove newline character
    input[strcspn(input, "\n")] = '\0';

    // Split the command into arguments
    int i = 0;
    args[i] = strtok(input, " ");

    while (args[i] != NULL && i < MAX_ARGS - 1)
    {
        i++;
        args[i] = strtok(NULL, " ");
    }

    // Create child process
    pid_t pid = fork();

    if (pid < 0)
    {
        printf("Fork failed!\n");
        return 1;
    }
```

```
    // Child Process
    else if (pid == 0)
    {
        printf("\n===== Child Process =====\n");
        printf("Child PID  : %d\n", getpid());
        printf("Parent PID : %d\n", getppid());

        // Execute Linux command
        execvp(args[0], args);

        // Executes only if execvp fails
        perror("Execution Failed");
        exit(1);
    }

    // Parent Process
    else
    {
        printf("\n===== Parent Process =====\n");
        printf("Parent PID : %d\n", getpid());
        printf("Child PID  : %d\n", pid);

        // Wait for child process
        wait(NULL);

        printf("\nChild process completed successfully.\n");
    }

    return 0;
}
```

Commands Used

Compile the program:

gcc Week1_Task.c -o Week1_Task

Run the program:

./Week1_Task

Example Input:

uname -a

Linux Commands Studied

1. uname

Command:

uname -a

Purpose: Displays information about the Linux kernel, operating system, and architecture.

---

2. lscpu

Command:

lscpu

Purpose: Displays CPU architecture, processor model, number of cores, threads, and cache information.

---

3. lsblk

Command:

lsblk

Purpose: Lists storage devices, partitions, and mounted file systems.

---

4. ps

Command:

ps

Purpose: Displays the currently running processes.

---

5. top

Command:

top

Purpose: Displays real-time information about CPU usage, memory usage, and active processes.

Observation

- The fork() system call successfully created a child process.
- The child process executed the Linux command using execvp().
- The parent process waited until the child process completed execution using wait().
- Different Process IDs (PIDs) were observed for the parent and child processes.
- Linux commands such as uname, lscpu, lsblk, ps, and top provided useful information about the operating system and hardware resources.

Result

The program was executed successfully. The child process executed the user-entered Linux command using execvp(), while the parent process waited using wait(). The Process IDs of both the parent and child processes were displayed correctly, demonstrating Linux process creation, command execution, and process synchronization.